Trabajo Práctico Obligatorio de Programación Orientada a Objetos

Documento de análisis y diseño del sistema

Propuesta C: Sistema de movilidad compartida (bicicletas y monopatines)

Etapa 1 · Entrega: 25/10/2026

1. Propuesta seleccionada e integrantes del equipo

El equipo eligió la Propuesta C: un servicio público de bicicletas y monopatines eléctricos compartidos que un municipio pone en marcha, con vehículos distribuidos en estaciones fijas de la ciudad. Los usuarios retiran y devuelven vehículos, pagan con un saldo prepago, y el personal del servicio redistribuye los vehículos y los mantiene.

Integrante	Legajo	Usuario de GitHub
[Completar]	[Completar]	[Completar]
[Completar]	[Completar]	[Completar]
[Completar]	[Completar]	[Completar]

Repositorio: [URL del repositorio de GitHub]

Tag de la entrega: etapa-1

Archivos de los diagramas: carpeta /docs/uml del repositorio (archivos .drawio editables e imágenes exportadas).

2. Requisitos y supuestos

2.1 Actores

Actor	Rol en el sistema
Usuario	Se registra, recarga saldo, se suscribe a un plan, retira y devuelve vehículos y reporta fallas.
Operador	Redistribuye vehículos entre estaciones, reporta fallas, genera órdenes de mantenimiento y registra ajustes de viajes.
Técnico de mantenimiento	Ejecuta las órdenes de mantenimiento y las cierra.
Administrador	Emite reportes y define tarifas y promociones.

2.2 Requerimientos funcionales

- RF1. Registrar usuarios y gestionar su cuenta: saldo, medios de pago y recargas.
- RF2. Retirar un vehículo de una estación e iniciar el viaje.
- RF3. Devolver un vehículo en una estación, cerrar el viaje y registrar inicio, fin y recorrido.
- RF4. Calcular la tarifa según el tipo de vehículo, el tiempo de uso y las promociones vigentes.
- RF5. Gestionar el estado de los vehículos y generar órdenes de mantenimiento a partir de los reportes de falla.
- RF6. Redistribuir vehículos entre estaciones cuando una queda vacía o saturada.

- RF7. Emitir reportes: viajes por usuario, uso por estación y vehículos con más fallas.

2.3 Reglas de negocio

Código	Regla
RN1	Solo retira quien tenga saldo mayor o igual al mínimo (equivalente a 30 minutos de uso) y ningún viaje abierto.
RN2	Si al cerrar un viaje el saldo queda negativo, el usuario queda inhabilitado hasta regularizarlo.
RN3	Solo un vehículo disponible puede retirarse. Estados: disponible, en uso, en reparación, batería baja y fuera de servicio.
RN4	Los eléctricos con batería menor al 15 % pasan a batería baja y no se ofrecen. Las bicicletas comunes no tienen batería.
RN5	Un vehículo con 3 reportes de falla en 30 días pasa a mantenimiento obligatorio.
RN6	Una orden de mantenimiento no se cierra sin registrar técnico, fecha y tareas realizadas.
RN7	Cada estación tiene capacidad fija. No se puede devolver un vehículo en una estación llena ni fuera de una estación.
RN8	Si la estación está llena, el usuario dispone de 10 minutos adicionales sin cargo para llegar a otra cercana.
RN9	Una estación con menos del 20 % o más del 90 % de ocupación genera una alerta de redistribución para el operador.
RN10	Tarifa = cargo de desbloqueo + minutos × precio por minuto según el tipo de vehículo (bicicleta común $200 + $20; bicicleta eléctrica $300 + $45; monopatín $300 + $50).
RN11	Un retiro cancelado dentro de los 2 minutos no tiene cargo.
RN12	Tiempo máximo de viaje: 2 horas; pasado el límite el minuto se cobra con 50 % de recargo. A las 24 horas sin devolver, el vehículo se reporta como perdido y se aplica un cargo fijo.
RN13	Promociones: el primer viaje de cada usuario es gratis hasta 15 minutos; el plan mensual incluye 20 viajes de hasta 30 minutos. No se acumulan: se aplica la más favorable para el usuario.
RN14	Un viaje cerrado no se modifica. Las correcciones las hace el operador con una nota de ajuste.

2.4 Supuestos adoptados

El enunciado deja algunos puntos abiertos. Para resolverlos el equipo adoptó los siguientes supuestos:

- **S1.** Saldo mínimo para retirar = cargo de desbloqueo + 30 minutos × precio por minuto del tipo de vehículo.
- **S2.** Los minutos se redondean hacia arriba.
- **S3.** Cargo fijo por vehículo perdido: $5.000 (valor de referencia, configurable).
- **S4.** En las promociones, el tiempo que excede el beneficio (15 o 30 minutos) se cobra a tarifa normal. "Más favorable" significa la que da menor costo final.
- **S5.** Un viaje cancelado es el que se devuelve en la misma estación dentro de los 2 minutos posteriores al retiro.
- **S6.** El plan mensual se renueva cada mes y el contador de viajes incluidos se reinicia.
- **S7.** La persistencia se hará en archivos (CSV o JSON) y la interfaz será por consola con menú.
- **S8.** El nivel de batería se actualiza al devolver el vehículo, informando el nivel resultante.
- **S9.** Las coordenadas son latitud y longitud de tipo double. La distancia del viaje se calcula con los puntos del recorrido.
- **S10.** Cada falla reportada se asocia a un solo vehículo. Para RN5 cuentan los reportes de los últimos 30 días corridos.

- **S11.** El técnico que cierra una orden es el que tiene asignada.
- **S12.** Los valores numéricos de las reglas (tarifas, umbrales, cargos) son de referencia y quedan como constantes o configuración, fáciles de ajustar.

3. Casos de uso

3.1 Diagrama de casos de uso

El diagrama muestra los cuatro actores y los trece casos de uso del sistema. "Validar usuario habilitado" es un paso incluido en CU3; CU5 está incluido en CU4; CU6 extiende a CU5 cuando existe una promoción vigente, y CU8 extiende a CU7 cuando un vehículo llega a los 3 reportes de falla en 30 días.

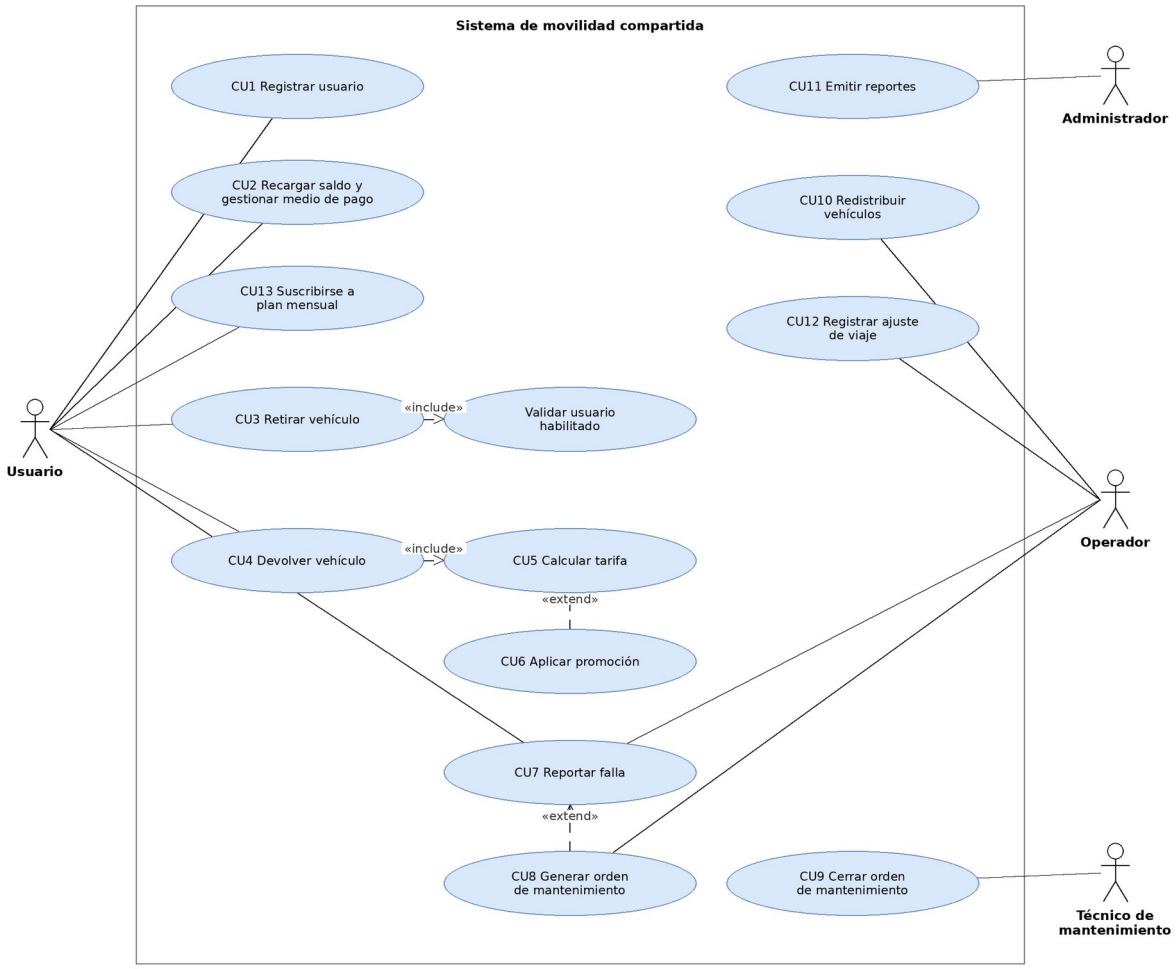


Figura 1. Diagrama de casos de uso del sistema de movilidad compartida.

3.2 Descripción de los casos de uso

CU1 – Registrar usuario

Actor: Usuario

Precondición: Documento no registrado previamente.

Flujo principal: 1) El usuario ingresa nombre, documento y correo. 2) El sistema valida los datos y que el documento no exista. 3) Crea el Usuario con su Cuenta (saldo 0) y la fecha de alta. 4) Confirma el registro.

Flujos alternativos: 2a) Documento ya registrado: se rechaza el alta. 2b) Datos inválidos: se solicita la corrección.

CU2 – Recargar saldo y gestionar medio de pago

Actor: Usuario

Precondición: Usuario registrado.

Flujo principal: 1) El usuario agrega un medio de pago (tarjeta o billetera virtual) o elige recargar. 2) Indica medio e importe. 3) El sistema procesa el pago con el medio. 4) La Cuenta registra una Recarga y actualiza el saldo. 5) Si el usuario estaba inhabilitado por deuda y el saldo queda en cero o más, se lo habilita.

Flujos alternativos: 3a) Pago rechazado: el saldo no cambia y se informa. 2a) Importe menor o igual a cero: se rechaza.

CU3 – Retirar vehículo

Actor: Usuario

Precondición: Usuario registrado; vehículo guardado en una estación. Incluye "Validar usuario habilitado".

Flujo principal: 1) El usuario elige estación y vehículo. 2) El sistema valida que el usuario esté habilitado, no tenga un viaje abierto y su saldo alcance el mínimo (RN1). 3) Verifica que el vehículo pueda retirarse (RN3, RN4). 4) Quita el vehículo de la estación, lo pasa a "en uso" y crea el Viaje en curso. 5) Informa el inicio del viaje.

Flujos alternativos: 2a) Usuario inhabilitado o con viaje abierto: se rechaza el retiro. 2b) Saldo insuficiente: se rechaza y se sugiere recargar. 3a) Vehículo no disponible o con batería menor al 15 %: se ofrece otro.

CU4 – Devolver vehículo

Actor: Usuario

Precondición: Viaje en curso. Incluye "Calcular tarifa".

Flujo principal: 1) El usuario indica la estación de devolución. 2) El sistema verifica que haya lugar (RN7). 3) Finaliza el viaje registrando el fin, la distancia y el recorrido. 4) Calcula la tarifa (CU5). 5) Debita la cuenta y registra el movimiento. 6) Deja el vehículo en la estación, actualiza kilómetros y batería, y lo marca disponible o con batería baja. 7) Si el saldo queda negativo, inhabilita al usuario (RN2).

Flujos alternativos: 2a) Estación llena: se informa y se otorgan 10 minutos sin cargo para ir a otra (RN8). 3a) Viaje de más de 2 horas: recargo del 50 % por minuto excedente (RN12). 3b) Más de 24 horas sin devolver: vehículo perdido y cargo fijo (RN12).

CU5 – Calcular tarifa

Actor: Sistema (incluido en CU4)

Precondición: Viaje con inicio y fin.

Flujo principal: 1) Obtiene la tarifa vigente del tipo de vehículo. 2) Calcula desbloqueo + minutos × precio, con recargo por exceso. 3) Evalúa las promociones aplicables (CU6) y elige la más favorable. 4) Devuelve el costo del viaje.

Flujos alternativos: 3a) Sin promociones aplicables: se cobra la tarifa normal.

CU6 – Aplicar promoción

Actor: Sistema (extiende CU5)

Precondición: Existe al menos una promoción vigente.

Flujo principal: 1) Para cada promoción vigente, el sistema verifica si aplica al viaje. 2) Calcula el descuento correspondiente. 3) Se conserva el menor costo final (RN13).

Flujos alternativos: 1a) El usuario no cumple las condiciones (por ejemplo, ya tuvo su primer viaje): la promoción no se aplica.

CU7 – Reportar falla

Actor: Usuario / Operador

Precondición: Vehículo existente.

Flujo principal: 1) El reportante indica el vehículo y una descripción. 2) El sistema registra el ReporteFalla con la fecha. 3) Si el vehículo acumula 3 reportes en 30 días, pasa a mantenimiento obligatorio y se avisa al operador (RN5).

Flujos alternativos: 1a) Vehículo inexistente: se informa el error.

CU8 – Generar orden de mantenimiento

Actor: Operador

Precondición: Hay reportes de falla sin orden asociada.

Flujo principal: 1) El operador elige los reportes de un vehículo y un técnico. 2) El sistema crea la OrdenMantenimiento con fecha de apertura. 3) Asigna el técnico y pasa el vehículo a "en reparación".

Flujos alternativos: 1a) Reportes de vehículos distintos: se rechaza. 1b) Técnico no disponible: se elige otro.

CU9 – Cerrar orden de mantenimiento

Actor: Técnico de mantenimiento

Precondición: Orden abierta y asignada al técnico.

Flujo principal: 1) El técnico registra las tareas realizadas y los repuestos usados. 2) Indica la fecha de cierre. 3) El sistema valida técnico, fecha y tareas (RN6) y cierra la orden. 4) El vehículo vuelve a "disponible".

Flujos alternativos: 3a) Faltan técnico, fecha o tareas: se impide el cierre.

CU10 – Redistribuir vehículos

Actor: Operador

Precondición: Alerta de redistribución activa (RN9).

Flujo principal: 1) El operador consulta las alertas. 2) Elige estación de origen, de destino y los vehículos a mover. 3) El sistema valida que haya lugar en el destino y que los vehículos estén disponibles. 4) Registra el Traslado y actualiza ambas estaciones.

Flujos alternativos: 3a) El destino no tiene lugar: se rechaza. 3b) Algún vehículo no está disponible: se excluye del traslado.

CU11 – Emitir reportes

Actor: Administrador

Precondición: Ninguna.

Flujo principal: 1) El administrador elige el reporte: viajes por usuario, uso por estación o vehículos con más fallas. 2) Indica los parámetros. 3) El sistema genera y muestra el reporte.

Flujos alternativos: 3a) No hay datos para los parámetros indicados: se informa el resultado vacío.

CU12 – Registrar ajuste de viaje

Actor: Operador

Precondición: Viaje cerrado.

Flujo principal: 1) El operador elige el viaje, el importe y el motivo. 2) El sistema crea la NotaAjuste asociada al viaje. 3) Registra el movimiento correspondiente en la cuenta del usuario. El viaje original no se modifica (RN14).

Flujos alternativos: 1a) El viaje sigue abierto: se rechaza el ajuste.

CU13 – Suscribirse a plan mensual

Actor: Usuario

Precondición: Usuario registrado.

Flujo principal: 1) El usuario elige un plan. 2) El sistema debita el precio del plan. 3) Crea la Suscripcion con vigencia de un mes y 20 viajes incluidos.

Flujos alternativos: 2a) Saldo insuficiente: se rechaza. 2b) Ya tiene una suscripción vigente: se informa.

4. Diagrama de clases inicial

El diagrama completo está en el archivo Clases_inicial.drawio (carpeta /docs/uml). Para que sea legible en este documento se lo presenta dividido en cuatro módulos; las clases con borde gris son referencias a clases que se detallan en otro módulo.

Jerarquías de herencia con sobreescritura de métodos: Vehiculo (abstracta) con BicicletaComun y VehiculoElectrico (abstracta), de la que heredan BicicletaElectrica y MonopatinElectrico; se sobreescriben getTipo() y puedeSerRetirado(). Personal (abstracta) con Operador, Tecnico y Administrador (getRol()). MovimientoCuenta (abstracta) con Recarga y Debito (getTipo()). MedioPago (abstracta) con Tarjeta y BilleteraVirtual (procesarPago()). Promocion (abstracta) con PromocionPrimerViaje y PromocionPlanMensual (aplicaA()).

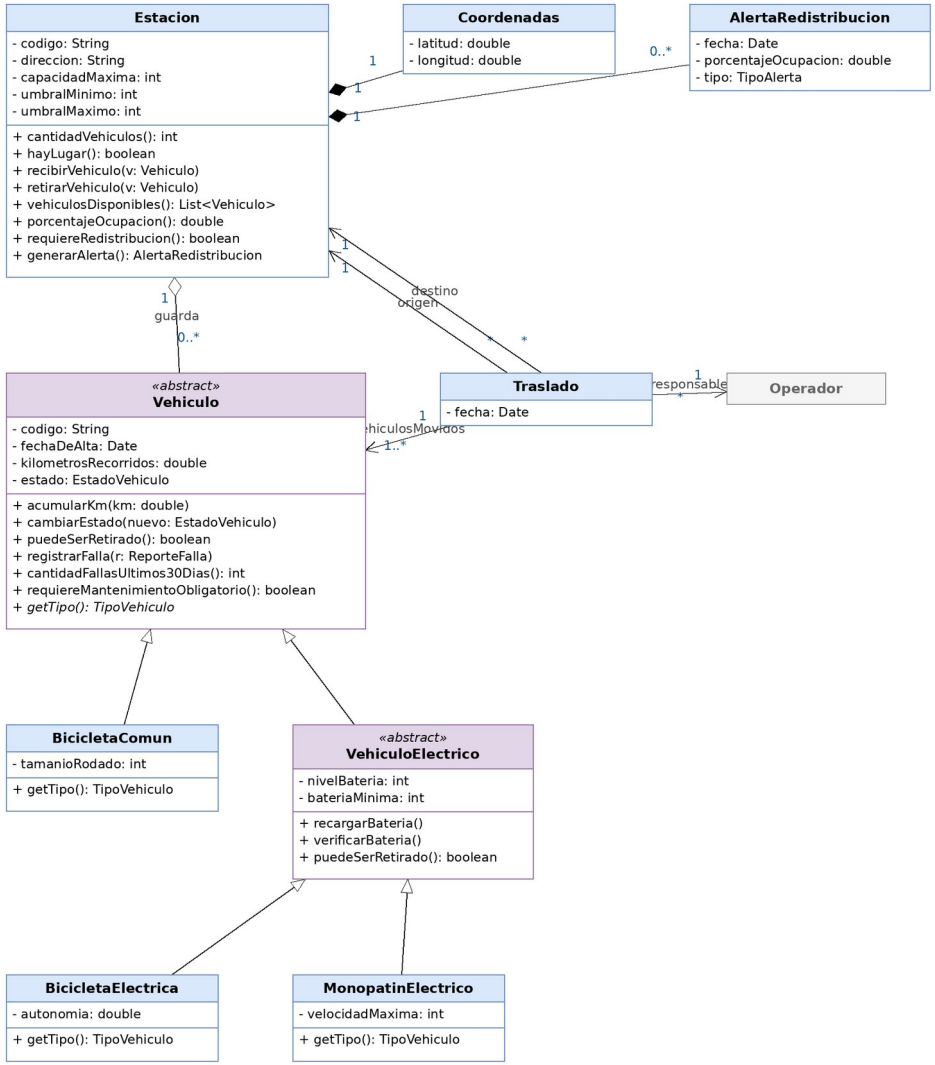


Figura 2. Diagrama de clases inicial – módulo de vehículos y estaciones.

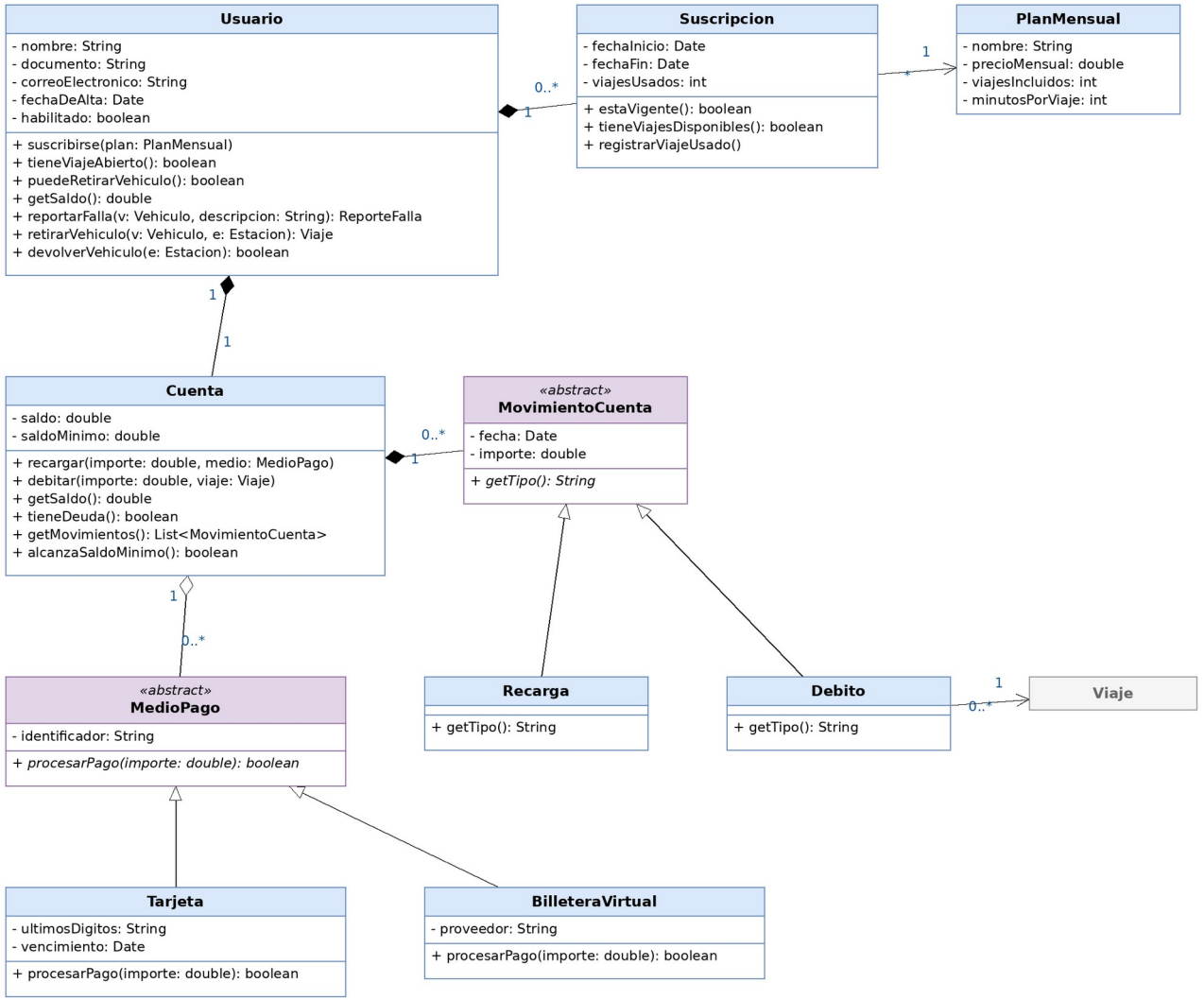


Figura 3. Diagrama de clases inicial – módulo de usuarios, cuenta y pagos.

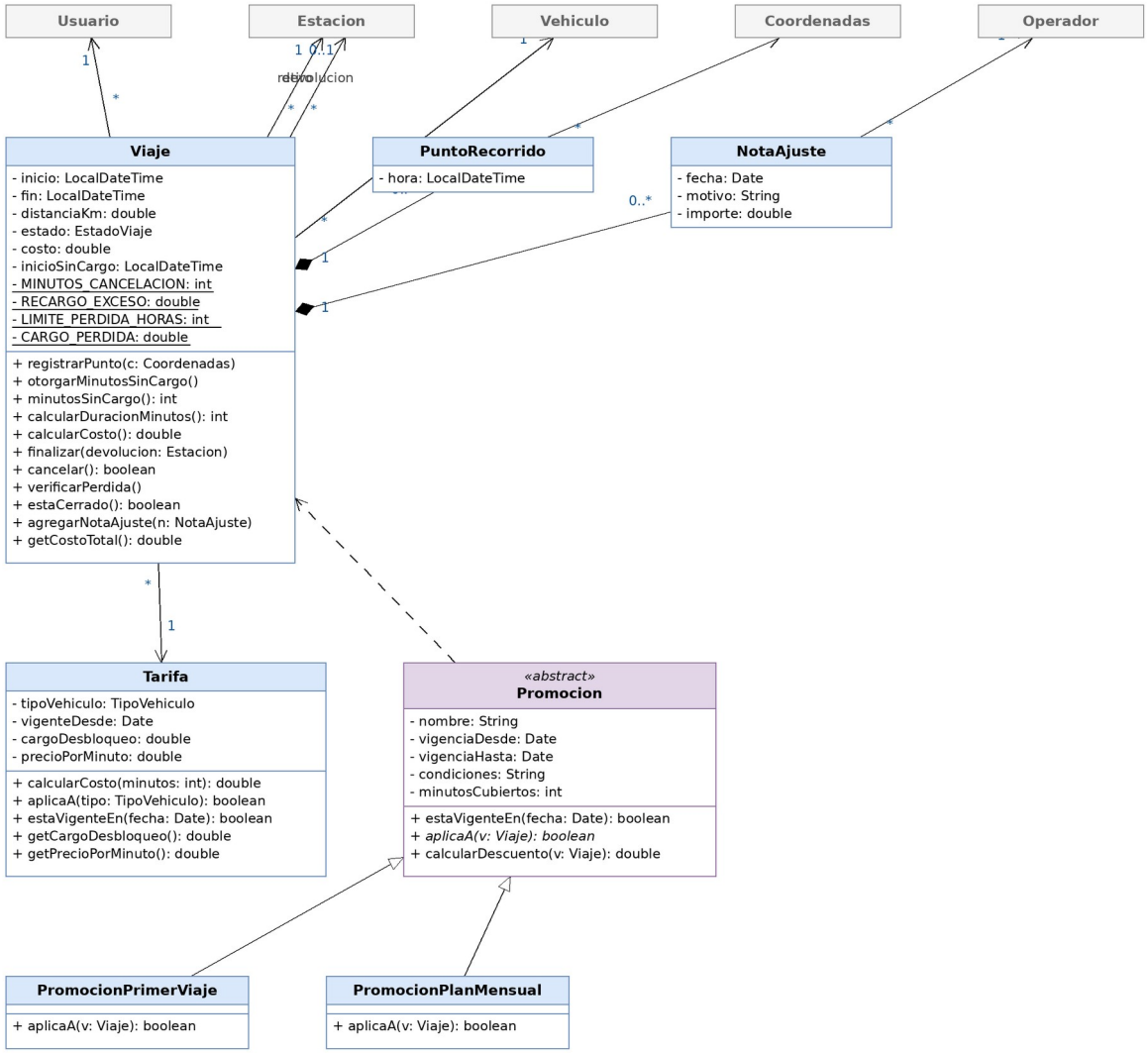


Figura 4. Diagrama de clases inicial – módulo de viajes, tarifas y promociones.

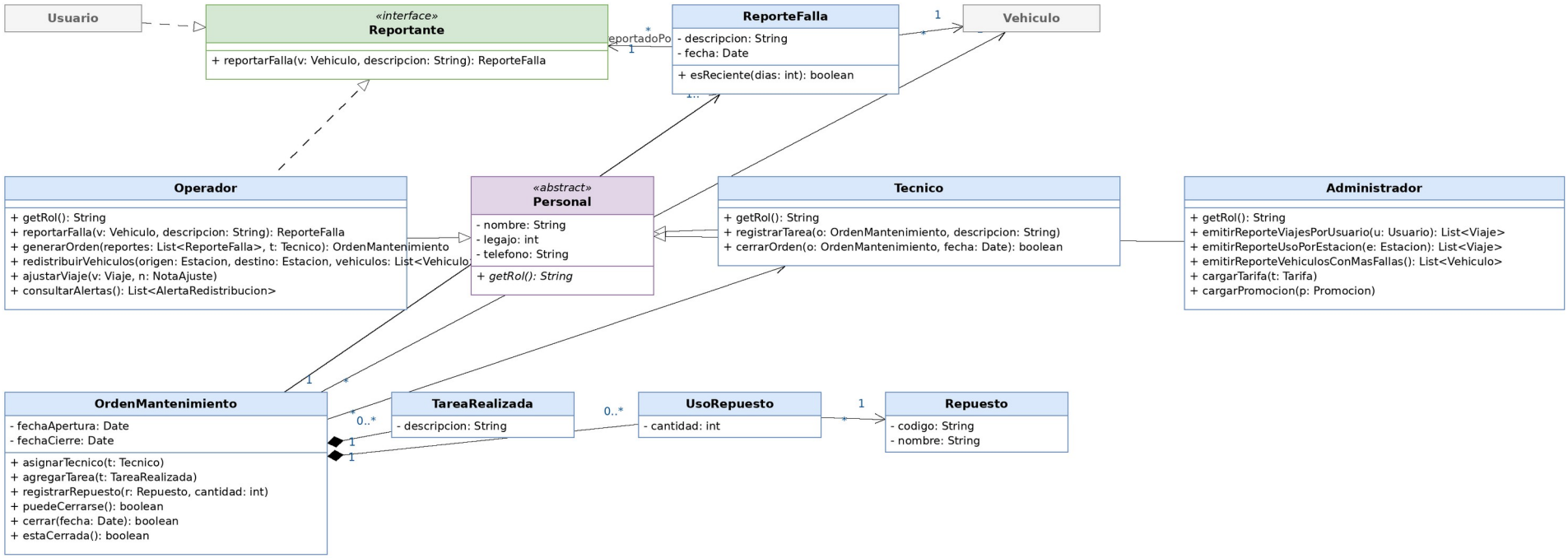


Figura 5. Diagrama de clases inicial – módulo de mantenimiento y personal.

5. Diagramas de secuencia

Se desarrollan dos casos de uso relevantes, ambos con flujos alternativos y de error: CU3 Retirar vehículo (rechazo por viaje abierto, inhabilitación o saldo insuficiente, y vehículo no disponible) y CU4 Devolver vehículo (estación llena, promociones y saldo negativo). Los mensajes usan los métodos del diagrama de clases inicial.

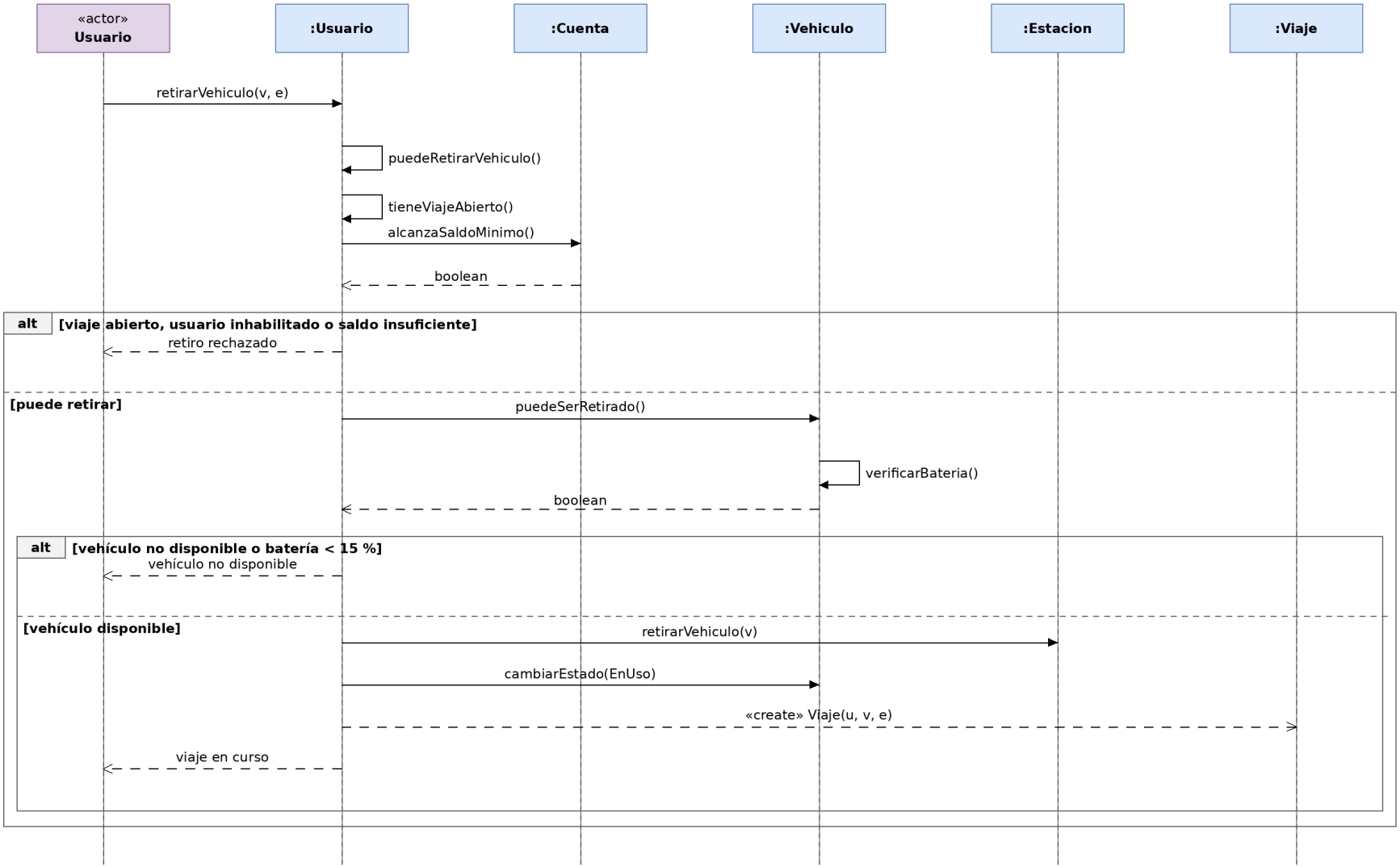


Figura 6. Diagrama de secuencia de CU3 – Retirar vehículo.

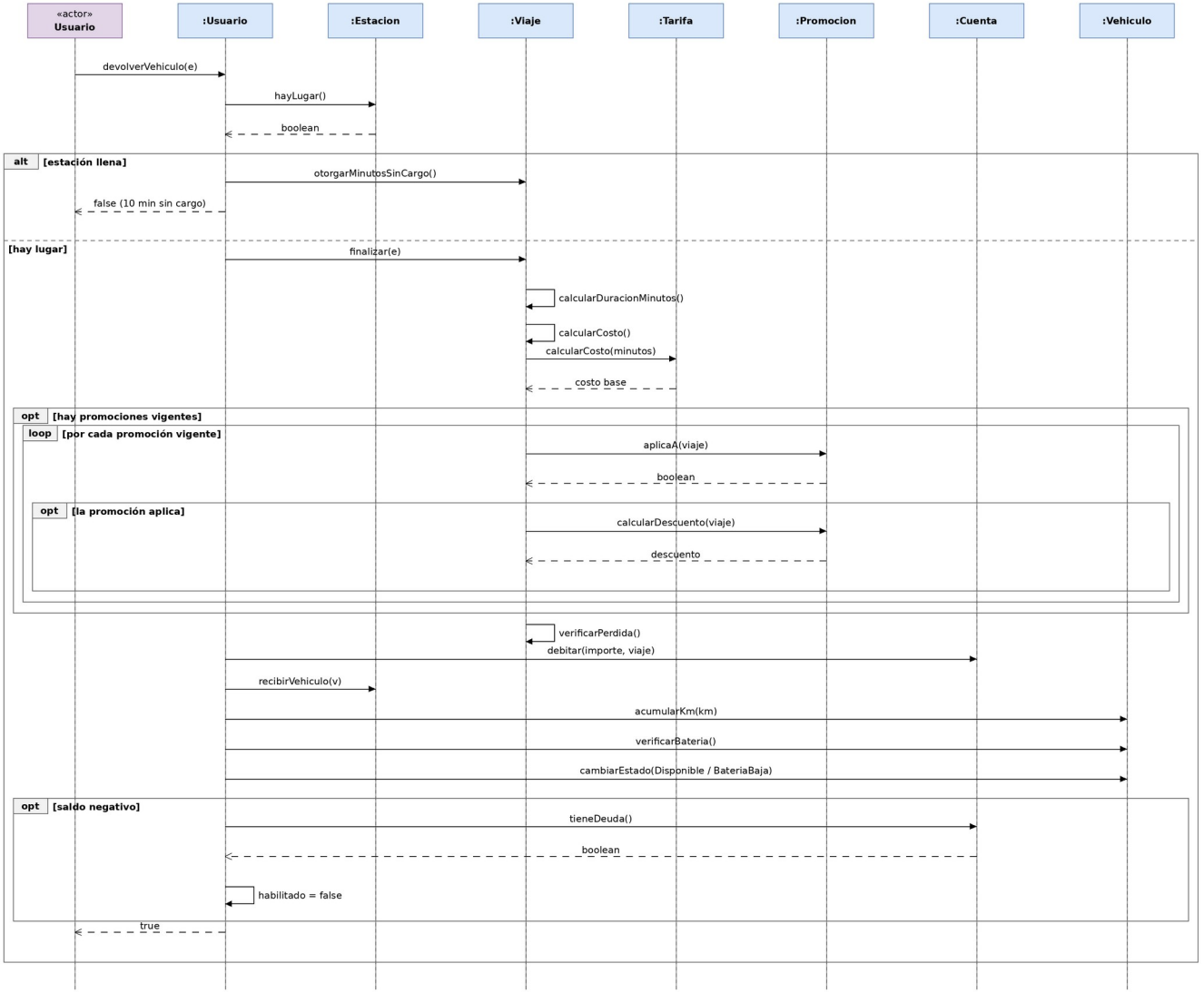


Figura 7. Diagrama de secuencia de CU4 – Devolver vehículo.

6. Aplicación de principios SOLID y patrones GRASP

Se revisó el diseño inicial y se detectaron tres problemas principales: (1) Viaje calcula su propio costo mezclando tarifas, promociones y recargos; (2) Usuario, Operador, Tecnico y Administrador tienen métodos que orquestan casos de uso completos, mezclando los datos de las personas con la lógica de negocio; y (3) no está definido cómo se persiste la información. A continuación se justifica cada principio y patrón elegido.

6.1 Principios SOLID

S – Principio de responsabilidad única

Problema detectado: Viaje guardaba los datos del viaje y además calculaba el costo: buscaba la tarifa, aplicaba el recargo por exceso, evaluaba las promociones y determinaba el cargo por pérdida. Un cambio en cualquiera de esas políticas obligaba a modificar la clase Viaje.

Decisión tomada: Se creó la clase CalculadorTarifa, que concentra el cálculo. Viaje conserva solo sus datos y su ciclo de vida (finalizar, cancelar, calcularDuracionMinutos) y recibe el resultado con registrarCosto(). Se eliminaron calcularCosto() y las constantes de recargo y cargo por pérdida de Viaje, y la asociación directa Viaje–Tarifa.

Clases donde se refleja: Viaje, CalculadorTarifa, Tarifa.

Ventaja: Cada clase tiene un único motivo de cambio. La política de precios se modifica y se prueba sin tocar ni crear viajes.

O – Principio abierto/cerrado

Problema detectado: En el diseño inicial el cálculo de costo debía preguntar de qué promoción se trataba (primer viaje, plan mensual) con condicionales; cada promoción nueva obligaba a modificar esa lógica.

Decisión tomada: Promocion es una clase abstracta con aplicaA(viaje) y calcularDescuento(viaje). CalculadorTarifa recorre su lista de promociones vigentes sin conocer los tipos concretos y se queda con el menor costo final (RN13). PromocionPrimerViaje y PromocionPlanMensual implementan la regla de cada una.

Clases donde se refleja: Promocion, PromocionPrimerViaje, PromocionPlanMensual, CalculadorTarifa.

Ventaja: Una promoción nueva (por ejemplo, descuento estudiantil) se agrega como una subclase más, sin modificar el calculador ni las demás promociones.

D – Principio de inversión de dependencias

Problema detectado: Los servicios que guardan y leen datos dependían de la forma concreta de persistir (archivos), lo que los ataba a un formato y dificultaba probarlos sin escribir archivos reales.

Decisión tomada: Se definió la interfaz Repositorio<T> con guardar, buscarPorId y listarTodos, y la implementación RepositorioArchivo<T> que usa archivos. ServicioViajes, ServicioMantenimiento, ServicioRedistribucion y ServicioReportes dependen de la interfaz y reciben la implementación por constructor.

Clases donde se refleja: Repositorio, RepositorioArchivo, ServicioViajes, ServicioMantenimiento, ServicioRedistribucion, ServicioReportes.

Ventaja: Las pruebas unitarias usan un repositorio en memoria, y se puede cambiar el formato de archivo (CSV o JSON) sin tocar la lógica de negocio.

6.2 Patrones GRASP

Experto

Problema detectado: Había que decidir qué clase responde preguntas como "¿hay lugar en la estación?" o "¿alcanza el saldo?". Si lo hace un servicio o el usuario, necesita conocer los datos internos de otras clases.

Decisión tomada: La responsabilidad se asigna a la clase que tiene la información: Estacion decide hayLugar(), porcentajeOcupacion() y requiereRedistribucion(); Cuenta decide alcanzaSaldoMinimo() y tieneDeuda(); Vehiculo decide puedeSerRetirado() y cantidadFallasUltimos30Dias(); Viaje calcula calcularDuracionMinutos().

Clases donde se refleja: Estacion, Cuenta, Vehiculo, Viaje.

Ventaja: Los datos quedan encapsulados y la lógica no se duplica en varias clases.

Creador

Problema detectado: Había que definir quién instancia los objetos del dominio.

Decisión tomada: ServicioViajes crea el Viaje (tiene el usuario, el vehículo y la estación, y registra los viajes); Cuenta crea sus Recarga y Debito porque contiene y registra los movimientos; Viaje crea sus PuntoRecorrido (composición); Estacion crea su AlertaRedistribucion en generarAlerta().

Clases donde se refleja: ServicioViajes, Cuenta, Viaje, Estacion.

Ventaja: La creación queda en la clase que contiene o usa a los objetos creados, lo que baja el acoplamiento y respeta el ciclo de vida de las composiciones.

Controlador

Problema detectado: Usuario, Operador, Tecnico y Administrador tenían métodos que orquestaban casos de uso completos (retirarVehiculo, devolverVehiculo, generarOrden, redistribuirVehiculos, emitir reportes). Eso mezclaba los datos de las personas con la coordinación de los casos de uso y obligaba a que la interfaz de consola conociera a las personas del dominio.

Decisión tomada: Se crearon cuatro controladores de casos de uso: ServicioViajes (CU3, CU4, CU12), ServicioMantenimiento (CU7, CU8, CU9), ServicioRedistribucion (CU10) y ServicioReportes (CU11). Reciben las solicitudes de la interfaz y coordinan a los objetos del dominio. Se quitaron esos métodos de las clases de personas.

Clases donde se refleja: ServicioViajes, ServicioMantenimiento, ServicioRedistribucion, ServicioReportes; se simplifican Usuario, Operador, Tecnico y Administrador.

Ventaja: La interfaz de consola (o una futura interfaz gráfica) solo se comunica con los controladores, y los casos de uso se pueden probar sin interfaz.

Polimorfismo

Problema detectado: El comportamiento varía según el tipo (el vehículo eléctrico debe verificar la batería y el común no; cada promoción y cada medio de pago funcionan distinto), lo que llevaría a condicionales por tipo repartidos por el código.

Decisión tomada: Se usan métodos sobreescritos en jerarquías: Vehiculo.puedeSerRetirado() y getTipo() (VehiculoElectrico agrega la verificación de batería), Promocion.aplicaA(), MedioPago.procesarPago(), MovimientoCuenta.getTipo() y Personal.getRol().

Clases donde se refleja: Vehiculo y sus subclases, Promocion y sus subclases, MedioPago, MovimientoCuenta, Personal.

Ventaja: Estacion y los servicios tratan a todos los vehículos, promociones y medios de pago por igual, y se pueden agregar tipos nuevos sin modificar el código que los usa.

7. Diagrama de clases final y justificación de los cambios

El diagrama de clases final incorpora las decisiones de la sección 6. El diagrama completo está en el archivo Clases_final.drawio (carpeta /docs/uml); en este documento se lo presenta por módulos en las figuras 8 a 12. Los módulos de vehículos y estaciones, y de pagos y mantenimiento que no se mencionan en la tabla no sufrieron cambios.

7.1 Cambios respecto del diseño inicial

Cambio	Motivo	Principio o patrón
Se agrega CalculadorTarifa. Viaje pierde calcularCosto(), getCostoTotal(), RECARGO_EXCESO y CARGO_PERDIDA, y gana registrarCosto() y getCosto(). Se elimina la asociación Viaje–Tarifa.	El cálculo de precios no es responsabilidad del viaje; el calculador selecciona la tarifa y las promociones.	SRP, Experto
CalculadorTarifa se asocia con Tarifa y Promocion; Promocion sigue siendo abstracta con sus dos subclases.	Permitir nuevas promociones sin modificar código existente.	OCP, Polimorfismo
Se agregan ServicioViajes, ServicioMantenimiento, ServicioRedistribucion y ServicioReportes.	Coordinar los casos de uso fuera de las clases de personas.	Controlador, SRP
Se quitan de Usuario retirarVehiculo() y devolverVehiculo(); de Operador generarOrden(), redistribuirVehiculos(), ajustarViaje() y consultarAlertas(); de Tecnico registrarTarea() y cerrarOrden(); de Administrador los reportes, cargarTarifa() y cargarPromocion().	Esa lógica pasa a los controladores y a OrdenMantenimiento.	Controlador, SRP
Usuario gana el atributo viajeActual (asociación 0..1 con Viaje) y los métodos estaHabilitado(), inhabilitar(), habilitar(), asignarViaje() y liberarViaje().	Que Usuario pueda responder tieneViajeAbierto() con sus propios datos y que la inhabilitación por saldo negativo (RN2) sea un método explícito.	Experto
Se agregan la interfaz Repositorio<T> y la implementación RepositorioArchivo<T>.	Persistir en archivos sin atar la lógica al formato.	DIP
Se agrega la jerarquía NegocioException con SaldoInsuficienteException, VehiculoNoDisponibleException, EstacionLlenaException, RetiroNoPermitidoException y OrdenIncompletaException.	Informar las violaciones de reglas de negocio con excepciones propias lanzadas con throw/throws, en lugar de códigos de retorno.	Requisito técnico de excepciones
Cuenta crea sus movimientos y Estacion crea sus alertas (sin cambios de estructura).	Se explicita la asignación de la creación.	Creador

7.2 Relación con los diagramas de secuencia

Los diagramas de secuencia de la sección 5 muestran el flujo de los casos de uso CU3 y CU4 con los mensajes del diseño inicial, donde el objeto Usuario recibe retirarVehiculo() y devolverVehiculo(). En el diseño final esos dos mensajes los recibe ServicioViajes, que envía a los mismos objetos (Cuenta, Vehiculo, Estacion, Viaje, Tarifa y Promocion) los mismos mensajes del diagrama, con estos cambios: el cálculo de costo lo hace CalculadorTarifa y las violaciones de reglas se informan lanzando las excepciones de negocio. Las reglas de negocio y el orden de los pasos no cambian.

7.3 Diagramas de clases finales por módulo

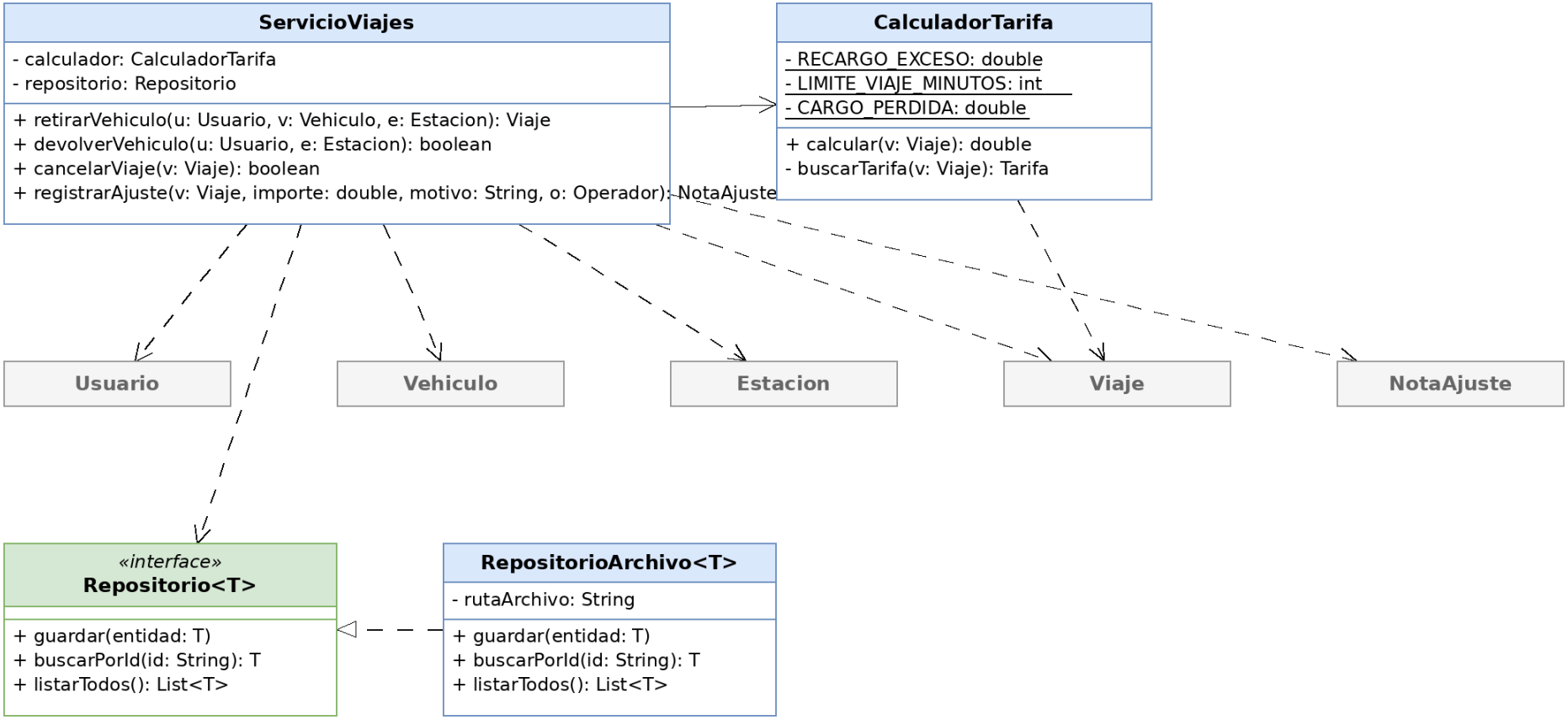


Figura 8. Diagrama de clases final – ServicioViajes, CalculadorTarifa y persistencia.

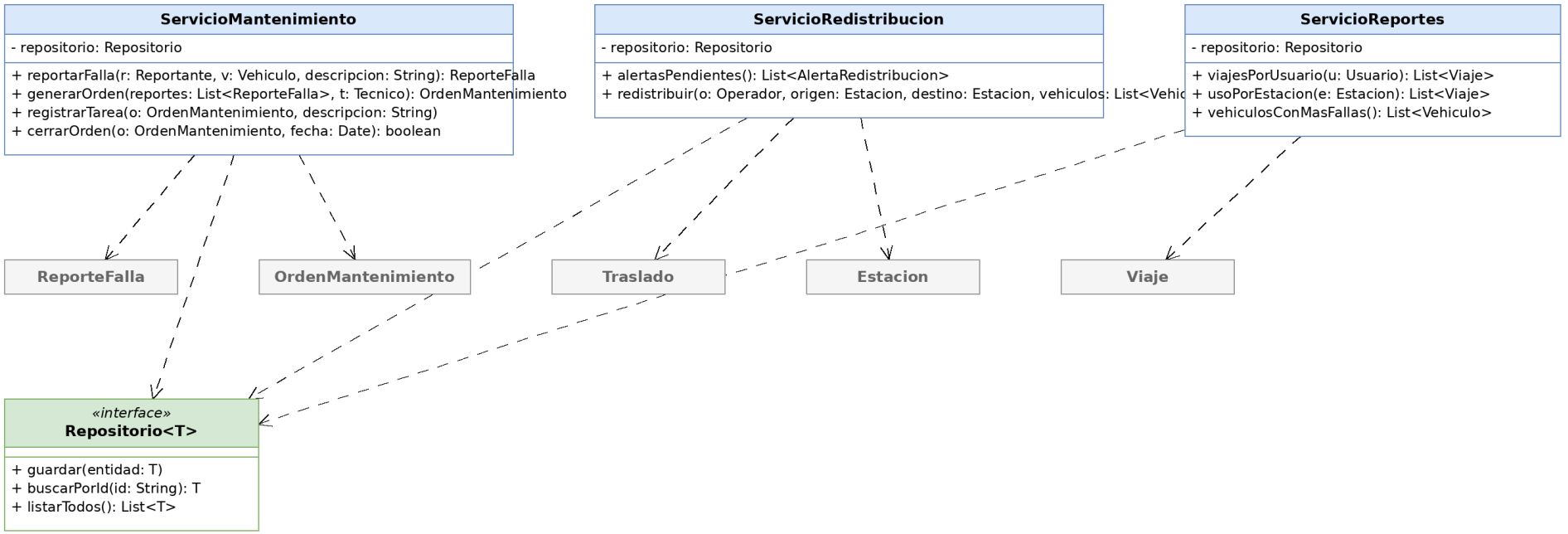


Figura 9. Diagrama de clases final – controladores de mantenimiento, redistribución y reportes.

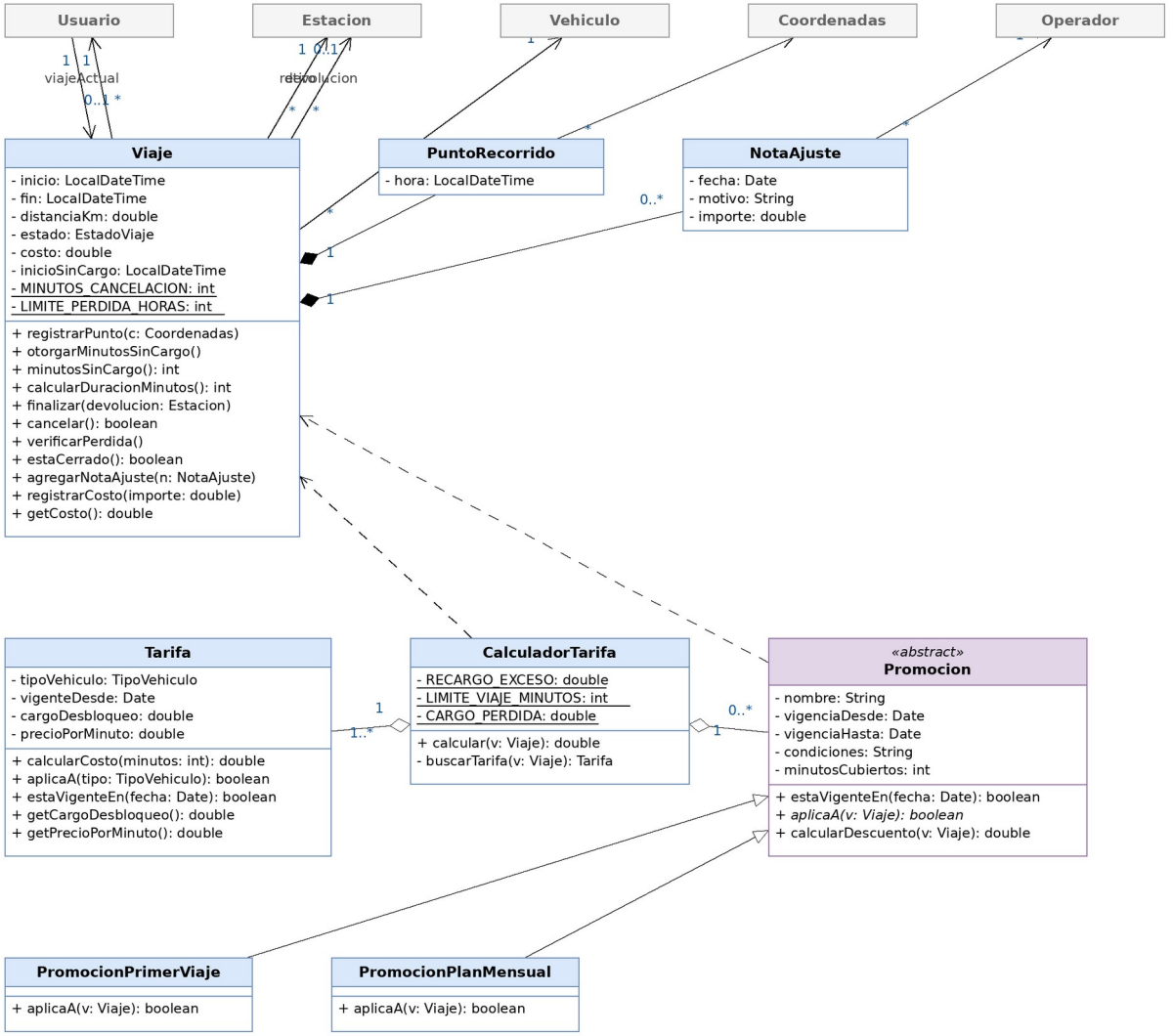


Figura 10. Diagrama de clases final – viajes, tarifas y promociones.

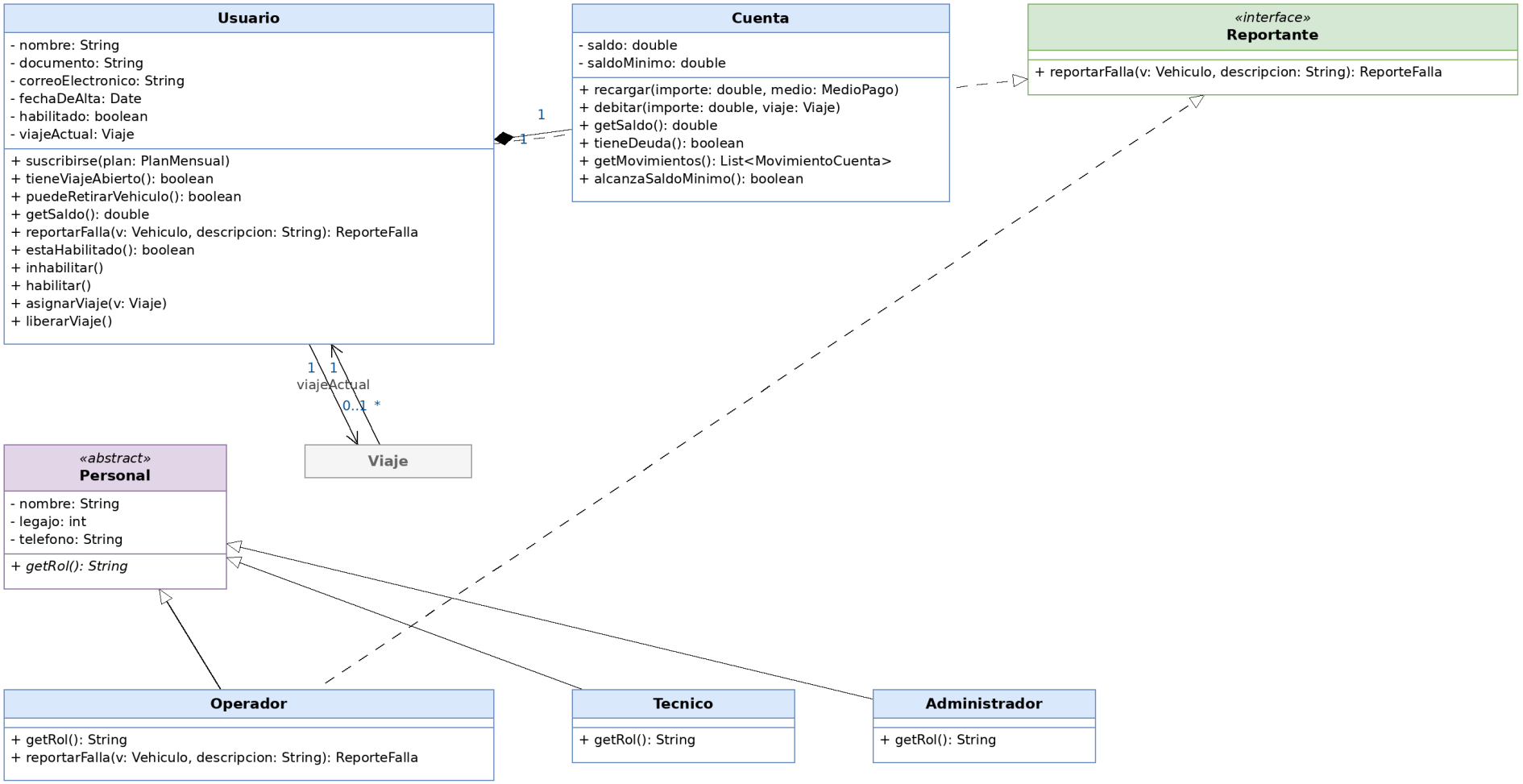


Figura 11. Diagrama de clases final – usuario, cuenta y personal.

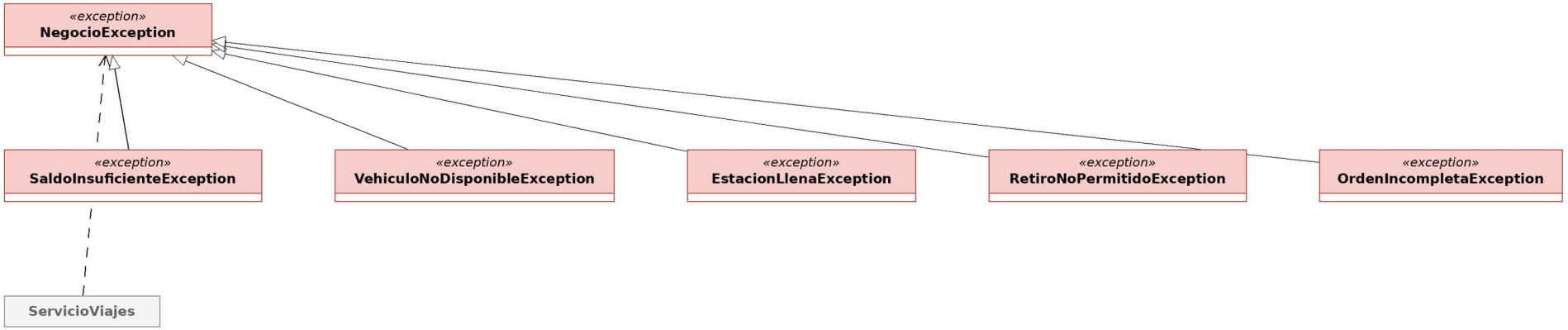


Figura 12. Diagrama de clases final – excepciones de negocio.